
SIMTiX

A GPU-Inspired RISC-V SIMT Accelerator
with an INT8/FP16 AI-Inference Extension

From programmable SIMT core to an FPGA-proven,
end-to-end quantized neural-network inference engine

Author: Krishna Gorai

Programme: M.Tech. — Computer Architecture, IIT Mandi

Platform: Xilinx Zynq UltraScale+ ZCU104 (xczu7ev)

M.Tech. Thesis Progress Presentation

1. Motivation & objectives
2. SIMTiX at a glance
3. Top-level architecture
4. SIMT execution model
5. Microarchitecture & latency hiding
6. Memory system & register files
7. Instruction-set architecture
8. Floating-point subsystem
9. Timing-closure contribution (100 MHz)
10. AI extension: pdot8 + operators
11. End-to-end MNIST inference
12. FPGA implementation results
13. Verification & design-space study
14. Contributions, status & future work

The gap

General-purpose CPUs are inefficient on the **data-parallel** workloads that dominate graphics and machine learning. Industrial GPUs solve this but are closed, large, and not reproducible for research or teaching.

Objective. Design, verify, and implement an **open, compact, GPU-inspired SIMT accelerator** around the RISC-V ISA that is:

- **Programmable** — runs ordinary compiled RISC-V kernels across many threads;
- **Complete** — integer, floating-point (FP32/FP16), and quantized-INT8 paths;
- **FPGA-proven** — a real bitstream on the ZCU104 that closes timing at 100 MHz;
- **Demonstrably an AI engine** — runs a real quantized network end-to-end and gets the right answers.

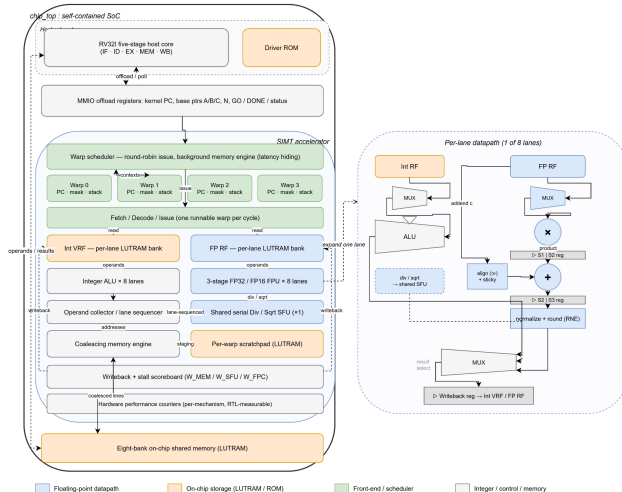
Key specifications

ISA	RV32IMF + Zfh (FP16) + custom pc
Organisation	8 lanes $\times$ 4 hardware warps
Pipeline	2-stage fetch/execute, in-order
Scheduler	round-robin, latency-hiding
Divergence	true SIMT reconvergence stack
Reg. files	per-lane LUTRAM (int + FP)
Memory	coalescing engine, 8-bank shared SRA
Target	xczu7ev-ffvc1156-2-e (ZCU104)

Proven capabilities

- Full **FP32 + FP16** datapath: add/mul, pipelined **FMA**, div/sqrt
- Custom **INT8 dot-product** instruction on a DSP engine
- Quantized operator library: GEMM, GEMV, requantize
- **End-to-end MNIST**: 64/64 bit-exact on hardware
- **100 MHz** closed with a clean-DRC bitstream
- >116 000 bit-exact self-checks across the suite

Top-Level Architecture — chip_top



RV32I host offloads kernels to the accelerator via MMIO; both share on-chip LUTRAM.

SIMT Execution Model

- A **warp** = 8 threads (one per lane) issuing a **single instruction** in lockstep.
- 4 warps resident; the **round-robin scheduler** picks one runnable warp per cycle — one warp's compute hides another warp's memory latency.
- **Control divergence** handled by a per-warp **reconvergence stack**: on a divergent branch the taken/not-taken paths execute serially under an active mask, then re-converge at the immediate post-dominator.
- **Coalescing**: a warp's 8 lane addresses that fall in one line become a **single** memory transaction (broadcast / contiguous), not 8.

Why it hides latency

Resource	Shared by
Fetch / decode / issue	all warps
8 integer ALUs	the issued warp
8 FP units (FMA)	the issued warp
Coalescing mem engine	one op in flight
Div/Sqrt SFU	all lanes (shared)

A long-latency op **parks** on its engine and the warp yields; other warps keep issuing.

Microarchitecture — Park-and-Resume Latency Hiding

The 2-stage front-end never stalls on a long op. Each long-latency class has a dedicated **background engine**; the issuing warp parks until the engine signals completion via a **stall scoreboard**.

Engine	Handles	Style
W_MEM	loads/stores (coalesced)	one line-transfer in flight
W_MUL	RV32M mul (16×16 DSP decomposition)	per-lane, multi-cycle
W_DOT	custom pdot8 INT8 dot product	DSP-pipelined, off critical path
W_SFU	FP divide / square-root	shared serial core, one lane at a time
W_FPC	pipelined FP add/mul/ FMA	3-stage, single-slot completion

Design principle

Keep the DSP- and FP-heavy blocks **off the fetch→execute critical path**. This is exactly what lets an 8-lane FP+INT8 fabric still close 100 MHz.

Register files in LUTRAM

- Integer VRF + FP RF: **one distributed-RAM bank per lane**, addressed by {warp, reg}.
- Async read, sync write — no block RAM spent on register storage.
- FP16 is **NaN-boxed** into the low 16 bits of the same 32-bit f-register $\Rightarrow$ one physical file serves FP32 and FP16.

On-chip memory

- **8-bank shared memory** (LUTRAM) — CPU-written kernels/weights are visible to the accelerator.
- Per-warp **scratchpad** for fast intra-warp reuse.
- **Coalescing memory engine** turns a warp's scattered lane accesses into whole-line transfers.
- AI demo adds a **256 KB block-RAM** weight store, streamed into LUTRAM on demand.

Storing register files and scratchpads in **LUTRAM** (not BRAM) is a deliberate choice: BRAM is reserved for the large weight store, and the design keeps flip-flop pressure low.

Kernels are compiled with a stock `-march=rv32imf` (+ Zfh) toolchain — no compiler patches. One custom opcode is added for AI.

Extension	Instructions implemented	Notes
RV32I	lui/auipc, jal/jalr, branches, loads/stores, ALU-imm, ALU-reg, ecall, csr	full base integer set
RV32M	mul (low 32 bits)	index / addressing
RV32F	add, sub, mul, fmadd/fmsub/fnmsub/fnmadd , div, sqrt, sgnj, min/max, cmp, cvt, mv, flw/fsw	single-precision
Zfh	half-precision variants of every F op + cvt.s.h	NaN-boxed FP16
Custom	pdot8 / pdot8u / pdot8su (custom-0 opcode 0x0B, 3 signedness variants)	INT8 4-way dot product

Floating-Point Subsystem

- **8 FP units** — one `simt_fpu` per lane, time-shared by the 4 warps (not 32 units).
- Unified datapath: the same multiply+add hardware serves `fadd`, `fmul`, and fused `fmadd` family.
- **FP16 support** by NaN-boxing + **widen** → **compute** → **narrow**, so FP16 reuses the FP32 core exactly.
- Multi-cycle **div/sqrt** on a shared serial SFU (folded from 8 per-lane cores to 1, saving area & de-congesting the fabric).
- IEEE-754 handling: subnormal flush-to-zero, correct NaN/Inf, round-to-nearest-even.

Pipelined FMA (3 stages)

S1: product + special-case detect

S2: 128-bit align + add → magnitude

S3: normalize + round (RNE)

Uniform 3-cycle latency; shallow ops ride a bypass register. This split is what moved the FP critical path **off the FMA**.

FP kernels shipped

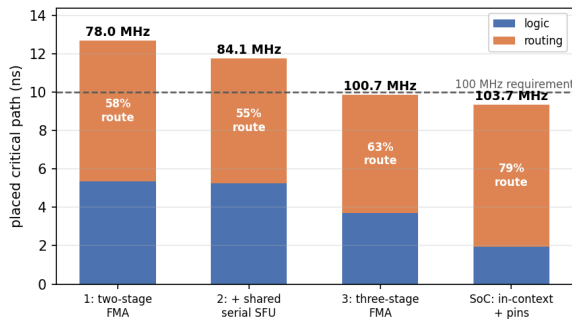
`fgemm`, `fsoftmax`, `fsaxpy(16)`, `fnorm`,
`fdot`

Timing-Closure Contribution — Reaching 100 MHz

An FP fabric is $2.5\times$ the integer area and **routing-bound**, not logic-bound. The thesis diagnoses this and fixes it in stages:

Step	Placed Fmax
2-stage FMA (M15)	78.0 MHz
+ shared serial SFU (M16)	84.1 MHz
3-stage FMA (M17)	100.7 MHz
SoC in-context + real pins	103.7 MHz

The bars split each critical path into **logic** vs **routing** delay — the honest finding is that congestion, not logic depth, was the limiter.



AI Extension — the pdot8 Custom Instruction

What it computes

$$rd = \sum_{i=0}^3 \text{ext}(rs1.\text{byte}[i]) \times \text{ext}(rs2.\text{byte}[i])$$

Four packed INT8 MACs in one instruction.
R-type, custom-0 opcode 0x0B, funct7=0.

Three signedness variants (funct3)

pdot8	S×S	weights × weights
pdot8u	U×U	activations × activations
pdot8su	S×U	weights × activations (inference)

- Emitted via `.insn` — **no compiler patch**, inert to existing kernels.
- Executes on the `W_DOT` DSP-pipelined background engine (4 DSPs/lane, $8 \times 4 = 32$ DSPs) — **off the critical path**.
- Non-accumulating (2-read-port VRF); the kernel accumulates with a separate add.
- The **only** custom opcode in the whole project — everything else is standard RISC-V.

- `qgemm` — INT8 GEMM tile ($M=N=8$, $K=16$): one warp/row, one lane/col; the reduction runs 4-at-a-time on `pdot8`.
- `qgemv` — INT8 GEMV, one lane per output; parameterized K, N so the **same kernel** runs both MLP layers.
- `requant_relu` — INT32→INT8 rescale + ReLU, reusing the FPU (branchless `fmadd+clamp`), so **no new hardware**.

Layouts are chosen so both operand streams **coalesce** and each stage's output is already the next stage's input format.

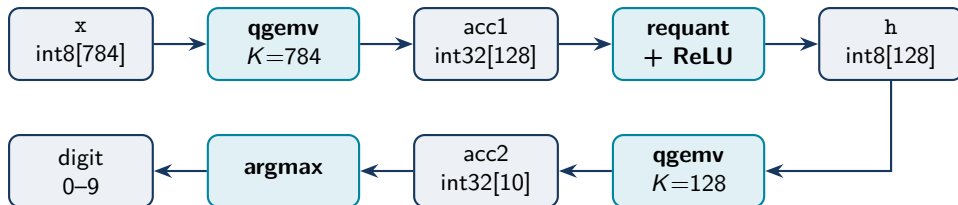
qgemm measured (8×8 , $K=16$)

Metric	Value
MACs	1024
Cycles	538
Throughput	1.90 MAC/cycle
Global-mem txns	72
Naïve (uncoalesced)	576
Traffic reduction	8×
Correctness	64/64 bit-exact

Coalescing matches the analytic model exactly.

End-to-End MNIST Inference

A pre-trained INT8 MLP ($784 \rightarrow 128 \rightarrow 10$) classifies MNIST digits using only the operators above:



Accuracy (numpy-trained, quantized)

FP32 test accuracy	96.82%
INT8 test accuracy	96.53%
Quantization drop	0.29%

Hardware correctness

vs. integer golden	64 / 64 bit-exact
on 64-image subset	98.4% (63/64)

On-Chip Demo — Host CPU Drives the Full Chip

The whole inference runs on the integrated chip_top, exactly as a deployed chip would:

- Weights live in a **256 KB block-RAM store**; the host CPU **streams** each weight tile into the 16 KB LUTRAM working memory.
- CPU launches `qgemv/requant_relu` per tile via MMIO, runs `argmax`, self-checks each digit against the on-chip golden.
- **The accelerator core is untouched** — BRAM's 1-cycle latency is hidden by a single CPU stall.
- **Result: 64/64 bit-exact** on real hardware.



The 64 test images, classified on-chip.

FPGA Implementation Results — PPA (xczu7ev / ZCU104)

Three representative builds of the complete chip, all on the ZCU104's Zynq UltraScale+:

Build	LUT	FF	DSP	BRAM
Integer baseline chip	34.3k	6.2k	24	0
FP-enabled chip (deployable, routed)	71.6k	–	40	0
AI MNIST chip (INT8 + BRAM)	83.5k	18.9k	72	64

Timing — all meet 100 MHz

Integer chip	100 MHz (+0.25 ns)
FP chip (routed)	103.7 MHz , 0 DRC
AI MNIST chip	100 MHz (+1.71 ns)

Power (vectorless)

Integer chip	0.96 W
FP chip	1.20 W
AI MNIST chip	1.36 W

FPGA Results — The AI MNIST Chip (Newest)

Utilization (OOC synthesis)

Resource	Used	Util. %
CLB LUTs	83,475	36.2
LUT as logic	69,247	30.1
LUT as memory	14,228	14.0
CLB registers	18,939	4.1
Block RAM (RAMB36)	64	20.5
DSP	72	4.2

- **First non-zero BRAM** in the project: the 256 KB weight store (2 Mbit) maps to **64 RAMB36** tiles, initialised bit-exact from the trained weights.
- **72 DSP** = FP fabric's 40 + pdot8's 32 INT8 MAC DSPs.
- **100 MHz met** with +1.71 ns slack; power **1.36 W**.
- Verified by a **post-synthesis gate-level simulation** of the synthesized netlist.

INT8 efficiency (measured, @100 MHz)

	Throughput	Efficiency
Measured (qgemm)	0.38 GOP/s	0.39 GOP/s/W
Datapath ceiling	6.4 GOP/s	6.6 GOP/s/W

For an 8-lane academic core these land honestly in the **GOPS** range (milli-TOPS) — the point is a *correct, coalesced, measured* baseline, not peak density.

The gap $\Rightarrow$ future work

The measured-vs-ceiling gap comes from the single-slot dot engine and per-warp dependency chains. The lever for real density is a **tensor / systolic engine** (Tier-3), not more SIMT tuning — qgemm defines the baseline it will be measured against.

Functional verification

- 14-testbench regression, all green.
- **>116,000 bit-exact** FP self-checks against a DPI reference (FP32 + FP16, all ops incl. FMA, div/sqrt).
- Golden-model checks for every quantized kernel and the full MNIST pipeline.

Silicon-accurate verification

- **Post-synthesis** gate-level simulation of the synthesized netlist.
- **Post-implementation** functional sim of the **routed** netlist — proves the real placed-and-routed hardware computes correctly.
- Clean-DRC bitstream with real ZCU104 pin constraints.

Tooling: Verilator (fast RTL regression) + Xilinx Vivado 2025.1 (synthesis, implementation, gate-level xsim); RISC-V GNU toolchain for kernels.

- **Lane count is not the Fmax lever.** Halving the fabric (4 vs 8 lanes) did **not** improve placed Fmax — the critical path is a *single* lane's FMA cone, independent of lane count. 8 lanes Pareto-dominates 4 (same Fmax, 2× throughput).
- **The FP fabric is interconnect-bound.** At ~31% device utilization the binding constraint is **routing / congestion**, not logic depth — confirmed by the shared-SFU de-congestion win (+6 MHz from identical logic).
- **FMA microarchitecture is the real lever.** A 3-stage FMA (attacking logic depth of the binding cone) is what finally closed 100 MHz; the path then moved off the FMA entirely onto the register-file read mux — the same limiter as the integer design.

A full diagnose → confirm → localize → fix arc, with every step measured on placed-and-routed hardware.

Key Contributions

1. A complete, open, **programmable RISC-V SIMT accelerator** (8 lanes $\times$ 4 warps) with true reconvergence, coalescing, and park-and-resume latency hiding.
2. A **FP32 + FP16** subsystem (NaN-boxed shared file, pipelined FMA, shared div/sqrt) that closes 100 MHz — with an honest, measured **timing-closure study** showing the design is interconnect-bound.
3. An **AI-inference extension**: a single custom INT8 dot-product instruction, a quantized operator library (GEMM/GEMV/requant), and an **end-to-end MNIST** demo that is **64/64 bit-exact on hardware**.
4. A **full RTL-to-bitstream flow** on the ZCU104 with silicon-accurate verification and a complete PPA characterization across integer, FP, and INT8+BRAM builds.

Current Status — Results Summary

Aspect	Result	Status
SIMT core (int)	8×4, coalescing, divergence	done
FP32 / FP16 subsystem	all ops, pipelined FMA, div/sqrt	done
Timing closure	100.7 MHz placed / 103.7 MHz SoC	done
Custom pdot8	INT8 4-way dot, 3 variants, DSP eng.	done
Quantized operators	qgemm/qgemv/requant, coalesced	done
End-to-end MNIST	96.53% INT8, 64/64 bit-exact on HW	done
AI chip PPA (+BRAM)	83.5k LUT / 64 BRAM / 72 DSP, 100 MHz	done
Verification	14-tb + 116k FP checks + gate-level	done

All milestones RTL-complete, FPGA-implemented, and verified; the M.Tech. thesis documents each build with its measured tables.

Near term

- Chip-level place-and-route of the MNIST build for a placed Fmax + bitstream.
- ZCU104 **board bring-up** of the on-chip demo.
- Shrink the register-file read mux (now the shared FP/int critical path).

Longer term

- **Tier-3 tensor / systolic engine** — output-stationary MAC array for real TOPS-class density (weight-sharing DSP packing).
- Dedicated integer requantize engine (FPU-less INT8 path).
- **Open-source sky130 GDS** (RTL-to-GDS) for an ASIC data point.

Thank you

SIMTiX — a RISC-V SIMT accelerator that runs a real quantized network
end-to-end on measured hardware

Questions & discussion

Krishna Gorai · M.Tech., IIT Mandi